

AI Development Instructions

AI Engineering Constitution (Summary Edition)

Bidra Platform

Version 1.0

Purpose

This document contains the mandatory engineering instructions that every AI agent must follow when generating code for Bidra.

It supplements the complete engineering documentation and should be treated as the highest-priority implementation guide.

The primary objective is to ensure that all AI-generated code is:

- Consistent
 - Secure
 - Testable
 - Maintainable
 - Production-ready
-

1. Understand Before Coding

Never begin implementation immediately.

Always:

1. Understand the feature request.
2. Identify affected modules.
3. Review existing code.
4. Reuse existing implementations.
5. Produce an implementation plan.
6. Only then write code.

Never guess requirements.

2. Preserve the Architecture

Bidra uses a **Modular Monolith Architecture**.

Never break module boundaries.

Each module owns:

- Business logic
- Database access
- Services
- Events
- Permissions

Modules communicate through services or domain events.

Never access another module's database tables directly.

3. Respect Layer Responsibilities

Controllers:

- HTTP only
- Validation
- Authentication
- Authorization

Never business logic.

Services:

- Business rules
- Workflows
- Transactions
- Domain validation

Repositories:

- Database persistence only

Never place business rules inside repositories.

4. Reuse Before Creating

Before creating:

- Component
- Service
- Hook
- DTO
- Repository
- Utility
- Event
- Validation schema

Search the codebase for an existing implementation.

Reuse whenever possible.

Avoid duplication.

5. Business Rules Live on the Server

The frontend must never calculate:

- Platform fees
- Time Credits
- Financial balances
- Permissions
- Reward eligibility
- Organization permissions

The backend is always authoritative.

6. Protect Immutable Data

The following records are append-only:

- Financial Ledger
- Time Credit Ledger
- Audit Logs
- Payment History

Never modify historical records.

Corrections must create new entries.

7. Database Rules

- PostgreSQL is the source of truth.
 - Use Prisma ORM.
 - Always create migrations.
 - Never edit applied migrations.
 - Use UUIDs internally.
 - Use Public IDs externally.
-

8. API Rules

Follow REST principles.

Use:

- GET
- POST
- PUT
- PATCH
- DELETE

Use nouns in URLs.

Example:

`/campaigns`

`/needs`

`/contributions`

Never expose internal UUIDs.

9. Validation

Validate all external input.

Including:

- API requests
- Query parameters
- Uploaded files
- Webhooks
- Environment variables

Never trust client input.

10. Authorization

Every protected endpoint must verify:

- Authentication
- Role
- Permission
- Organization membership
- Resource ownership

Never rely on frontend authorization.

11. Security

Never:

- Hardcode secrets
- Disable validation
- Skip permission checks
- Expose stack traces
- Log passwords
- Log API keys
- Log tokens

Follow Secure by Default.

12. Logging

Every important operation should include:

- Request ID
- User ID
- Organization ID (if applicable)
- Timestamp
- Action
- Duration

Never log sensitive data.

13. Audit Logging

Every sensitive action must create an immutable audit record.

Examples:

- Refunds
 - Permission changes
 - Organization verification
 - Time Credit adjustments
 - Platform settings
 - Reward redemption
 - Campaign publication
-

14. Transactions

Use database transactions for multi-step business operations.

Examples:

Donation

↓

Payment

↓

Ledger

↓

Contribution

↓

Need Progress

↓

Events

Everything succeeds together or rolls back together.

15. Domain Events

Emit events after successful business operations.

Examples:

- CampaignPublished
- PaymentSucceeded
- ContributionVerified
- VolunteerAttendanceVerified
- TimeCreditsAwarded
- RewardRedeemed

Events describe facts, not commands.

16. Frontend Rules

Use:

- Next.js App Router
- Server Components by default
- Client Components only when necessary
- React Hook Form
- Zod
- TanStack Query
- Zustand only for shared client state
- Tailwind CSS
- shadcn/ui

Never embed business logic in React components.

17. Backend Rules

Use:

- NestJS
- Prisma
- Services
- Repositories
- DTOs
- Domain Events

- BullMQ

Controllers remain thin.

Services own business logic.

18. Code Quality

Every implementation should:

- Be readable
- Be predictable
- Be modular
- Follow naming conventions
- Minimize duplication

Prefer clarity over cleverness.

19. Testing Requirements

Every feature must include appropriate tests.

At minimum:

- Unit tests
- Integration tests for business workflows
- Regression tests for bug fixes

Critical workflows require end-to-end tests.

20. Documentation

Whenever implementation changes:

Update:

- API documentation
- Database documentation
- Architecture documentation
- Relevant specifications

Documentation is part of the implementation.

21. Performance

Avoid:

- N+1 queries
- Loading unnecessary data
- Duplicate API requests
- Unbounded database queries

Optimize only after measuring.

22. Accessibility

All user interfaces must meet WCAG 2.2 AA.

Support:

- Keyboard navigation
 - Screen readers
 - Proper contrast
 - Semantic HTML
 - Focus management
-

23. AI Self-Review Checklist

Before completing any task, verify:

- ✓ Architecture respected
- ✓ Module boundaries preserved
- ✓ Existing code reused
- ✓ Validation implemented
- ✓ Authorization enforced
- ✓ Business logic in services

- ✓ Transactions used where required
 - ✓ Audit logging included
 - ✓ Tests written
 - ✓ Documentation updated
 - ✓ No duplicated logic
 - ✓ No security regressions
 - ✓ No performance regressions
-

24. AI Must Never

Never:

- Rewrite unrelated code.
 - Introduce new frameworks.
 - Duplicate business logic.
 - Bypass services.
 - Access another module's tables directly.
 - Expose internal UUIDs.
 - Modify immutable ledgers.
 - Modify audit history.
 - Remove tests to make builds pass.
 - Disable linting or type checking.
 - Ignore failing tests.
 - Skip validation.
 - Skip authorization.
-

25. Definition of Done

A feature is complete only when:

- Requirements are implemented.
- Code follows architecture.
- Tests pass.
- Documentation is updated.
- API is documented.
- Database migrations are included (if applicable).
- Logging is present.

- Audit logging is implemented where required.
 - Accessibility is maintained.
 - Performance is acceptable.
 - Security requirements are satisfied.
-

Final Principle

Build Bidra as though it will be maintained by engineers who have never seen this code before.

Every decision should make the platform easier to understand, easier to extend and safer to operate.

When multiple implementations are possible, choose the one that is simplest, most maintainable and most consistent with the existing architecture.